

ORGANISASI DAN ARSITEKTUR KOMPUTER

Pertemuan 6-7 :
Set Instruksi

KARAKTERISTIK DAN FUNGSI SET INSTRUKSI

- ⦿ Operasi dari CPU ditentukan oleh instruksi-instruksi yang dilaksanakan atau dijalankannya. Instruksi ini sering disebut sebagai instruksi mesin (*mechine instructions*) atau instruksi komputer (*computer instructions*).
- ⦿ Kumpulan dari instruksi-instruksi yang berbeda yang dapat dijalankan oleh CPU disebut set Instruksi (*Instruction Set*)

ELEMEN-ELEMEN DARI INSTRUKSI MESIN (SET INSTRUKSI)

- ⦿ Operation Code(opcode)
- ⦿ Source Operand Reference
- ⦿ Result Operand Reference
- ⦿ Next instruction Reference

Source dan *result Operands* dapat berupa salah Satu diantara tiga jenis berikut ini:

- Main or Virtual Memory
- CPU Register
- I/O Device

DESAIN SET INSTRUKSI

Desain setinstruksi merupakan masalah yang sangat komplek yang melibatkan banyak aspek, diantaranya adalah:

1. Kelengkapan set instruksi
2. Ortogonalitas (sifat independensi instruksi)
3. Kompatibilitas:

Selain ketiga aspek tersebut juga melibatkan hal-hal sebagai berikut:

1. Operation Repertoire

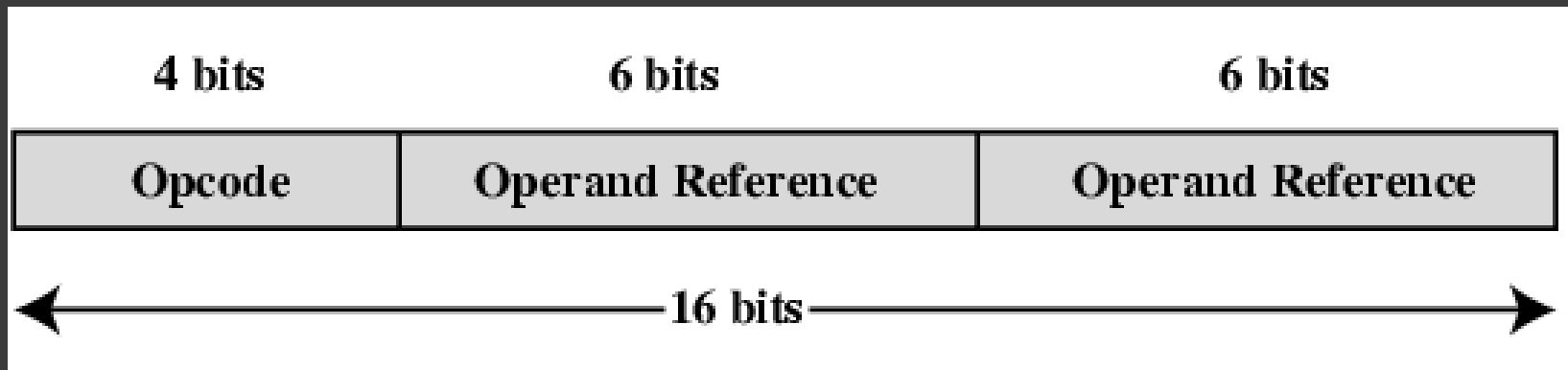
2. Data Types

3. Register

4. Addressing

FORMAT INSTRUKSI

Suatu instruksi terdiri dari beberapa field yang sesuai dengan elemen dalam instruksi tersebut. Layout dari suatu instruksi sering disebut sebagai Format Instruksi (*Instruction Format*).



Instruction Types

- ◉ Data processing
- ◉ Data storage (main memory)
 - ◉ Data movement (I/O)
 - ◉ Program flow control

Jenis-jenis Operand

- ◉ Addresses
- ◉ Numbers
- ◉ Characters
- ◉ Logical Data

JUMLAH ALAMAT

(NUMBER OF ADDRESSES)

Salah satu cara tradisional untuk menggambarkan arsitektur prosesor adalah dengan melihat jumlah alamat yang terkandung dalam setiap instruksinya.

Jumlah alamat maksimum yang mungkin diperlukan dalam sebuah instruksi:

- a. Tiga Alamat
- b. Dua Alamat
- c. Satu Alamat
- d. Nol Alamat

Number of Addresses (a)

⊙ 3 alamat

- Operand 1, Operand 2, Result
- $a = b + c;$
- May be a forth - next instruction (usually implicit)
- Not common
- Membutuhkan sangat banyak word untuk menyelesaikan

Number of Addresses (b)

⊙ 2 alamat

- One address doubles as operand and result
- $a = a + b$
- Reduces length of instruction
- Requires some extra work
 - Temporary storage to hold some results

Number of Addresses (c)

⦿ 1 Alamat

- Implicit second address
- Usually a register (accumulator)
- Common on early machines

Number of Addresses (d)

⦿ 0 (zero) alamat

- All addresses implicit
- Uses a stack
- e.g. push a
- push b
- add
- pop c
- $c = a + b$

Macam-macam instruksi menurut jumlah operasi yang dispesifikasikan

1. O –Address Instruction
2. 1 –AddreessInstruction.
3. N –Address Instruction
4. $M + N$ –Address Instruction

Tabel Model Pengalamatan

Mode	Algorithm	Principal Advantage	Principal Disadvantage
Immediate	Operand = A	No memory reference	Limited operand magnitude
Direct	EA = A	Simple	Limited address space
Indirect	EA = (A)	Large address space	Multiple memory references
Register	EA = R	No memory Reference	Limited address space
Register Indirect	EA = (R)	Large address space	Extra memory reference
Displace-ment	EA=A+(R)	flexibility	Complexity
Stack	EA=top of Stack	No memory Reference	Limited applicability

Tipe-Tipe Operasi

- ⦿ Data Transfer
- ⦿ Arithmetic
- ⦿ Logical
- ⦿ Conversion
- ⦿ I/O
- ⦿ System Control
- ⦿ Transfer of Control

Data Transfer

- ⊙ Specify
 - Source
 - Destination
 - Amount of data
- ⊙ May be different instructions for different movements
 - e.g. IBM 370
- ⊙ Or one instruction and different addresses
 - e.g. VAX

Arithmetic

- ⦿ Operasi aritmatik yang tersedia penjumlahan, pengurangan, pembagian dan perkalian
- ⦿ Signed Integer
- ⦿ Operasi tersedia dalam bentuk Floating point maupun fixed point
- ⦿ Operasi lain
 - Increment ($a++$)
 - Decrement ($a--$)
 - Negate ($-a$)

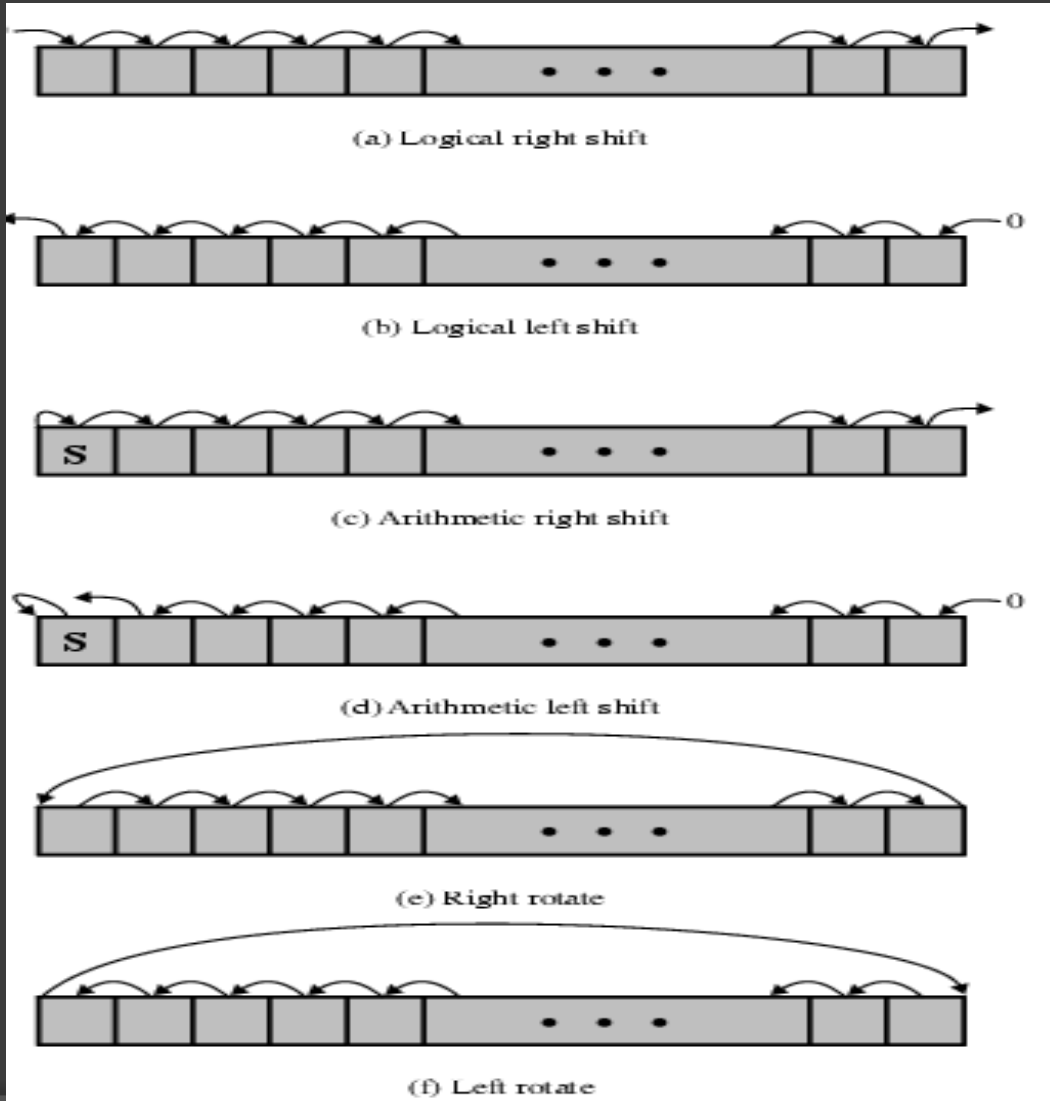
Logical

- ⦿ Menyediakan operasi boolean seperti : AND, OR, NOT
- ⦿ Mengoperasikan bit-bit word maupun alamat dalam membentuk operasi fungsi

Conversion

- Instruksi konversi adalah instruksi yang mengubah format data
- Misal : Binary to Decimal
Pengubahan kode 8 bit menjadi kode lain

Shift and Rotate Operations



Input/Output

- ⦿ Menggunakan instruksi spesifik
- ⦿ Digunakan untuk proses instruksi pemindahan data (memory mapped)
- ⦿ Digunakan untuk controlMay be done by a separate controller (DMA)

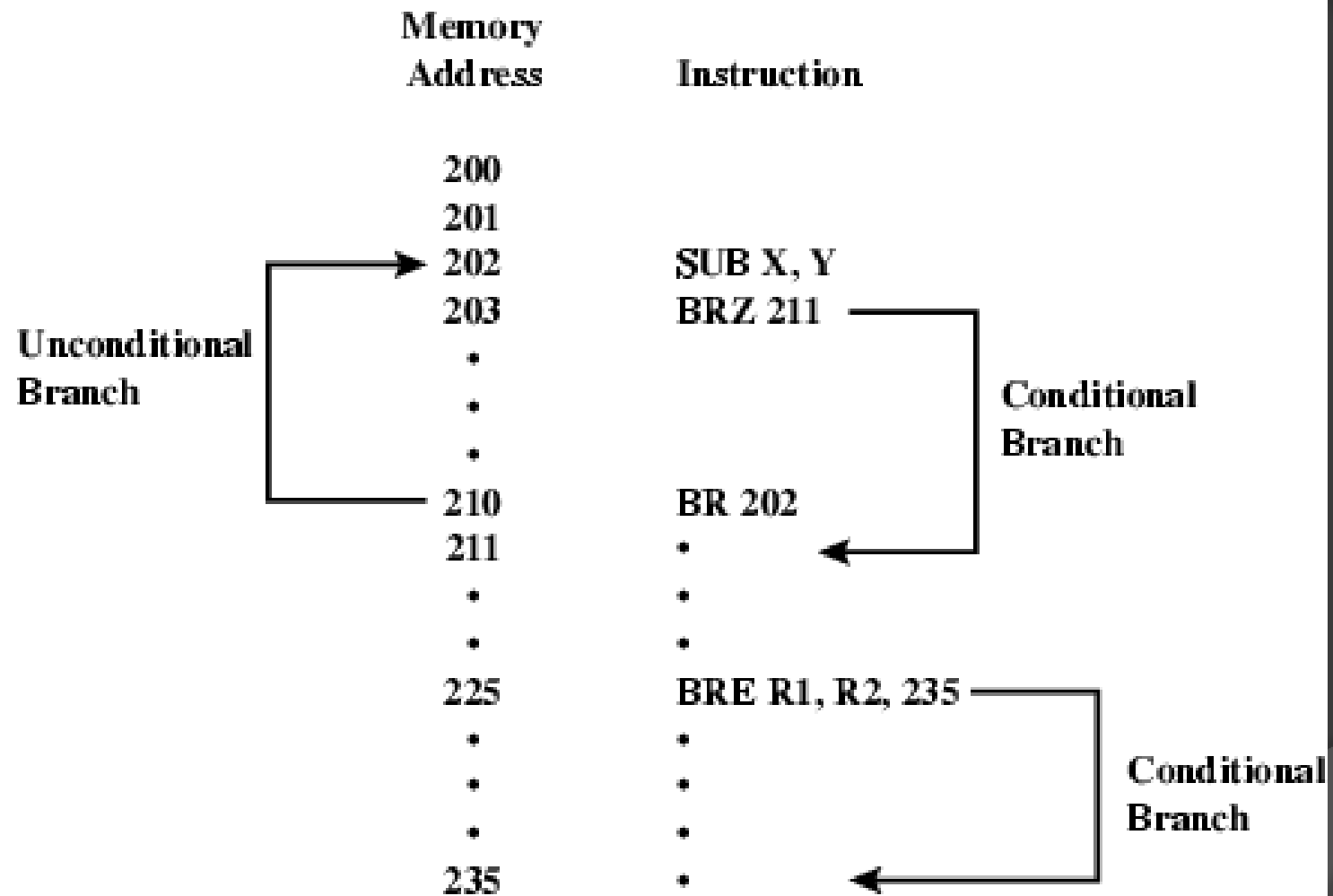
Systems Control

- ⦿ Merupakan instruksi khusus (*Privileged instructions*) yang hanya bisa dieksekusi disaat prosessor dalam keadaan khusus.
- ⦿ Contoh :
CPU membutuhkan specific state untuk
 - Ring 0 on 80386+
 - Kernel mode
- ⦿ Digunakan pada operating system

Transfer of Control

- ⦿ Instruksi yang dilakukan oleh instruksi itu sendiri
- ⦿ Branch (percabangan)
 - Misal : nilai x bercabang jika hasil adalah nol
- ⦿ Skip (lompat)
 - e.g. increment and skip if zero
 - ISZ Register1
 - Branch xxxx
 - ADD A
- ⦿ Subroutine call
 - c.f. interrupt call

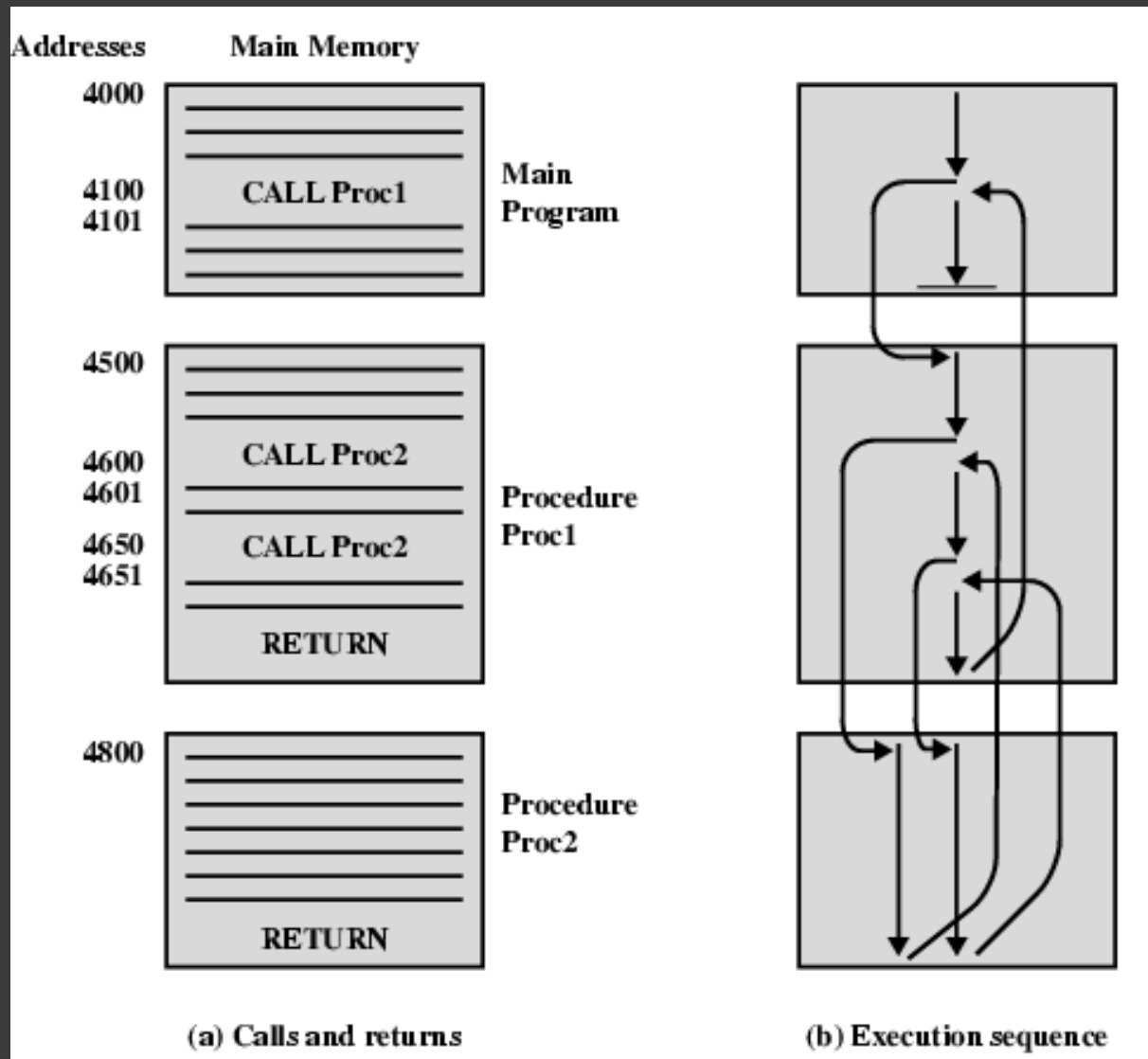
Branch Instruction



Subroutine call

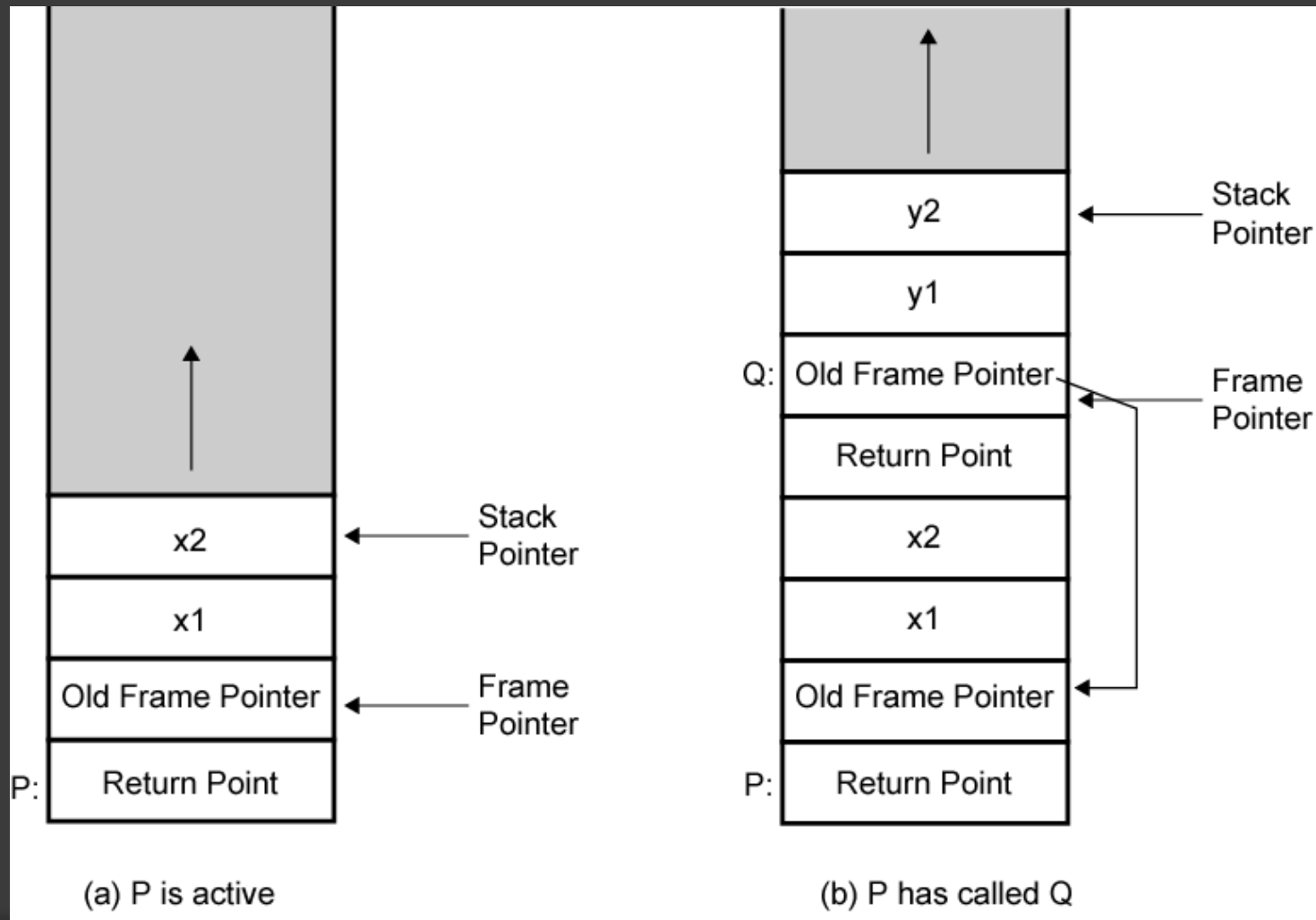
- ⦿ Program modular yang merupakan bagian program komputer yang lebih besar.
- ⦿ Berguna jika potongan program akan digunakan berulang kali.
- ⦿ Mekanisme melibatkan dua instruksibesar :
 - instruksi pemanggilan (call Instruction) yang bercabang ke subrutin
 - instruksi kembali (return instruction) yang mengembalikan program ke program pemanggilnya

Urutan eksekusi bersarang

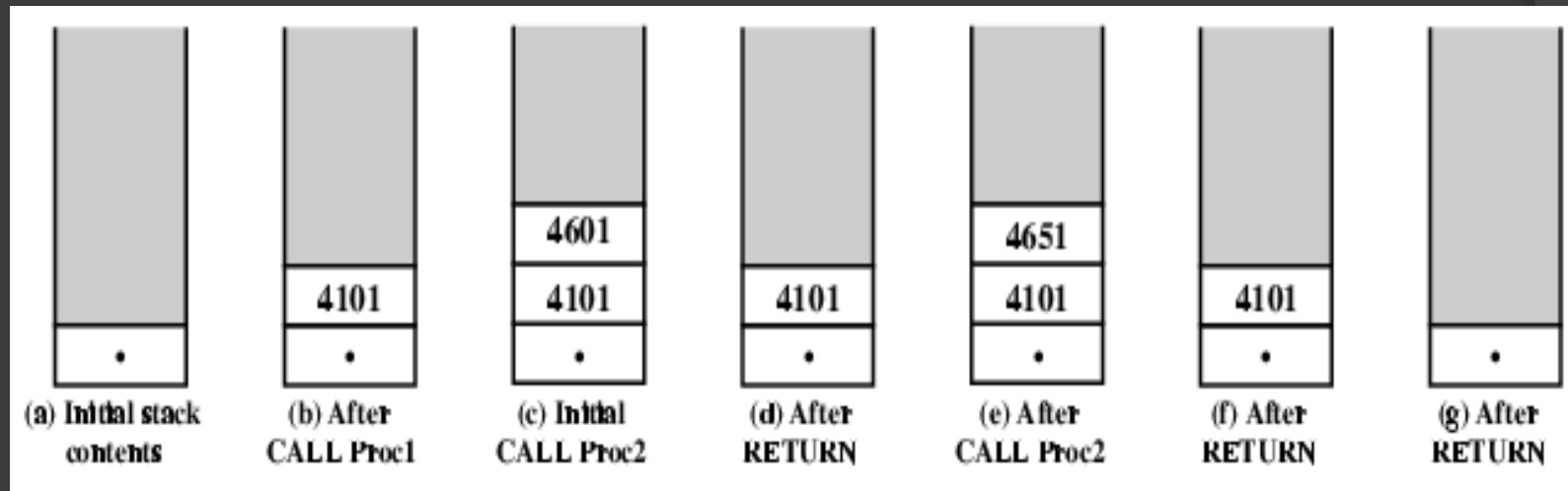


Contoh :

Contoh pemakaian Stak untuk masukan P dan Q



Contoh : Pemakaian Stack



- Sistem stack sebagai penyimpanan
- Saat CPU mengeksekusi perintah CALL
- maka CPU akan menaruh alamat pengembalian ke stack